pfi

Python for Fixed Income

COURSE 4 · MACHINE LEARNING

Course 4 Capstone

WEEK 3 · DAY 5

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The scenario: a public file and three questions

A colleague who covers emerging market corporates hands you a public file: ratios from the annual statements of Polish companies, each flagged by whether the company went bankrupt in the following year.

The question, in three parts

Which of three models separates them?

How well, on rows it never saw?

Where does it fail?

What the answer is not

Not a credit view on any company

Not a rating, not a credit decision

Not a probability of default going forward

Not advice to a lender or an investor

It is a description of a historical, anonymised dataset: what a model shows on it, and where it is wrong.

What you hand in: twelve deliverables

#	Deliverable	Where
1	mlkit.py complete, pytest passing	1
2	load_bankruptcy(path): read and check	2
3	The whole file described, hashed checks	3
4	prepare(frame): duplicates, then 80/20	4
5	Three models as unfitted pipelines	5
6	Thresholds for recall 0.70, out of fold	6

#	Deliverable	Where
7	The test rows, opened once	8
8	A final choice, with reasons	7 and 8
9	A results workbook, read back	9
10	What the model shows, and where it fails	10
11	One commit per file, then git tag course4	all
12	An AI-use log	10

The functions go into one module of your own, bankruptcy.py, which grows question by question.

The file: UCI Polish Companies Bankruptcy, 5th year

Real, public, anonymised

UCI Machine Learning Repository, dataset 365, CC BY 4.0. The page gives the file 5,910 statements, 410 of them from companies that went bankrupt one year later.

Bankrupt companies 2000 to 2012, the others 2007 to 2013, as the page says. One country: not US corporate credit.

Citation: Tomczak, S. (2016). Polish Companies Bankruptcy [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5F600.

The names, and the units

```
describe_feature("Attr2")
Attr2 (X2, total liabilities / total assets)
```

The file says Attr1 to Attr64; the page defines X1 to X64 and gives no mapping. Read Attr k as X k, and test it (question 3.10).

Not all ratios: X29 a logarithm, X55 an amount, nine features in days.

Ten sections, three sittings

First sitting

Your Complete Module

Load and Check the File

The Whole File, Described

Prepare: copies, then split

Check, describe, split

Second sitting

Three models

Thresholds for a Recall of 0.70

The choice

Model and choose

Third sitting

The Test Rows, Opened Once

The Results Workbook

What the model shows

Test once, report

Then the closing sections of every day: Excel Side by Side, Save the Day With Git, Bring It Home, and the AI check.

What makes it hard: nothing is planted

Missing values

a gap here and there, and some columns with many

Exact duplicate rows

copies of a whole statement, label and all

A rare outcome

a few percent of the statements

Wildly different scales

ratios near 0 or 1, and some in the millions

Section 3 describes the whole file before any split. Nothing computed there builds a feature or chooses anything.

"Stuck after trying?" blocks hold a decision, never a count. Make your own first.

Three roles of rows, and two leaks

Training rows

fit

every model, imputer, and transform

Folds of the training rows

choose

hyperparameters, thresholds, the model

Test rows

report

once, in section 8, after the choice

Leak 1: copies

A row and its twin on both sides of the split: a test row the model has seen. Drop exact duplicates before the split.

Leak 2: a fitted step

A median or quantile from all rows carries the test rows into training. Inside a pipeline, refitted in every fold, it cannot.

Three models, each a function

`build_logistic()`

- `SimpleImputer(median, indicators)`
- `QuantileTransformer(normal)`
- `LogisticRegression(balanced)`

`build_tree(...)`

- `SimpleImputer(median)`
- `DecisionTreeClassifier(balanced)`
- `pre-pruned: grid search`

`build_pruned_tree(...)`

- `SimpleImputer(median)`
- `DecisionTreeClassifier(balanced)`
- `post-pruned: ccp_alpha`

Each returns an unfitted pipeline: the function is the recipe, the caller decides which rows it learns from.

Trees split on order, so they need no scaling (day 9). The logistic model needs the quantile transform.

How out-of-fold probabilities work

The problem	The threshold must be chosen on rows the model did not fit, and there are no validation rows
Folds	cross_val_predict cuts the training rows into folds and fits a fresh copy on the other folds each time
One each	Every training row gets the probability from a model that never saw it
You control	The model, the folds, and the output: the probability of class 1
What comes out	One probability per training row; the chosen threshold is applied unchanged

The metric and the threshold, fixed in advance

Primary metric: average precision

Accuracy rewards flagging nothing. ROC AUC can look high while most flags are wrong. Average precision, the area under the precision-recall curve (day 8), starts at about the bankrupt share for a ranking at random.

Compared on the training folds.

Threshold: recall of at least 0.70

```
oof = cross_val_predict(model, X_train, y_train,
    cv=mlkit.cv_folds(), method="predict_proba")[:, 1]
mlkit.threshold_for_recall(y_train, oof, 0.70)
```

Out of fold, on the training rows; never the test rows. An illustration of the trade between catching and over-flagging, not a lending rule.

How the checks work

A check holds a short code, never the answer

```
check('Question 3.1', answer_3_1, 'a short code')
```

```
correct
```

```
not yet. You have ...
```

```
not attempted yet
```

Store the full number; check rounds to the places asked. Text without capitals. Several values in a tuple.

Decisions change the numbers

Checks after section 4 assume the course's decisions: duplicates dropped, the course's split, SEED. If a check says not yet and your code looks right, compare decisions, not numbers.

42 questions and 39 check lines, in the order an analyst works.

The choice, the test rows, the workbook, Git

Section 7

Choose

On the training folds alone. Reasons written down first.

Section 8

Test once

compare_models and each confusion matrix. The choice stands.

Section 9

Workbook

Four sheets with pandas, read back; Excel counts with COUNTIFS.

Git

Log

results.csv after the folds and after the test; git tag course4.

```
=COUNTIFS(B2:B...,1,C2:C...,">="&thresholds!B2)
```

Excel has no worksheet function for a logistic fit or a tree; the counting after them is Excel's job.

Every comparison: a chart and a table

Provided chart cells, sections 8 and 10

8.3	Confusion heatmaps: training rows, training folds, test rows
8.5	ROC and precision-recall curves, every model, thresholds marked
10	Every model on every set of rows: a styled table and bars
10.1	Coefficient bars beside the tree's importance bars

Each prints not attempted yet until the answers it draws from are stored.

The calls you write

```
plotkit.styled(cv_table, [...], "caption")
plotkit.bars(cv_table[[...]], "title",
             "Mean score (0 to 1)")
```

Questions 5.6, 6.3, and 8.1 ask for the comparison as a formatted table and as bars. A caption says what the rows are and where they were scored.

No chart from the work is shown in this video: the charts are yours to draw.

What a model's output may be called

Allowed names

model probability of bankruptcy in this file

the model's score

flagged at threshold t

Never called

a credit view on any company

a rating

an investment decision

a probability of default going forward

The description: the data, what was compared and where it was chosen, the test figures with counts, the ratios it leans on (association, not cause), where it fails, and what the data cannot support.

assert_descriptive is a reminder, not a proof. Read the page once more for meaning.

The log, the description, and the AI check

The AI-use log

What you asked, what you accepted or rejected, how you checked it. "none used" is valid. Rows and firm data stay out.

The description

300 to 500 words that describe what the model shows on this file and where it fails.

The AI check

One preparation step in the style of an assistant's answer. It runs, with one modelling bug. Write the test first.

Then, and only then, the solutions notebook.

Code that runs is not the same as code that is right.

What you can do now

- Fit on some rows, judge on rows the model never saw
- Read a coefficient, a p-value, and a threshold for what they are
- Pipelines that fit every step on training rows only
- Choose on training folds, open the test rows once
- Describe what a model shows and where it fails

Next keep your my_bondmath folder up to date, and bring it with you.